atlas

TECHNISCHES WHITEPAPER · NOVEMBER 2026

Capability-driven Automation

Von fachlichen Fähigkeiten zu nachweisbaren, autorisierten Worker-Operationen

FAKTENSTAND
7. November 2026

REPOSITORY-BASIS
main · 8c6d819b2591

AUSGABE
Neufassung 1.0 · DE-CH

Architektur- und Statusbericht · Developer Preview · Keine Zertifizierung

Inhalt

Management Summary

1. Ausgangslage und Problemstellung

2. Begriffe, Ebenen und Beweislast

3. Architekturüberblick und Ist-Zustand

4. Von der Facharchitektur zum technischen Vertrag

5. CDA0: deklarative Claims – umgesetzt

6. CDA1: semantischer Contract-Envelope – umgesetzt

7. CDA2: Governance, Conformance und Evidenz – teilweise umgesetzt

8. CDA3: verifizierte, aus der Quelle abgeleitete Worker-Claims – geplant

9. CDA4 und CDA5: Architekturgraph, Discovery und Impact – geplant

10. CDA6: BPMN-Binding und Deployment-Validierung – geplant

11. CDA7: Governed Execution – geplant

12. CDA8: kontrolliertes Rebinding – geplant, Entscheidung zurückgestellt

13. Sicherheit, Governance und Datenschutz

14. Durchgehendes Beispiel: Employee Onboarding

15. Programmstatus CDA0–CDA8 und Abhängigkeiten

16. Alternativen, Grenzen und Trade-offs

17. Offene Entscheidungen und konkrete nächste Schritte

18. Fazit

Quellen- und ADR-Verzeichnis

> **Leseschlüssel. Umgesetzt** bedeutet anhand aktueller Repository-Artefakte belegte Funktion. **Teilweise umgesetzt** bedeutet vorhandenen Code mit offenen wesentlichen Abnahmekriterien. **Geplant/offen** bedeutet dokumentiertes Zielbild ohne nachgewiesene Implementierung. **Einschätzung** bezeichnet die eigene fachliche Bewertung. Diese Unterscheidung gilt für jede einzelne Funktion und nicht pauschal für ein gesamtes Programm.

Management Summary

Capability-driven Automation (CDA) verbindet in atlas drei bisher getrennte Perspektiven: *Was* eine Organisation leisten muss (Business Capability), *wodurch* sie diese Leistung erbringt (BPMN-Prozess und Service Tasks) und *welche überprüfbare Operation* eine konfigurierte technische Integration tatsächlich ausführt. Ziel ist eine belegbare Verbindung von Geschäftsarchitektur, technischen Verträgen, Worker-Auswahl und Identitäts- sowie Berechtigungsprüfung. Die Architektur unterscheidet bewusst zwischen fachlicher Realisierung, deklarierter Fähigkeit, überprüfter Konformität und freigegebener Aktion.

Stand am 7. November 2026: Die Grundlagen sind konkreter als in der älteren Konzeptdarstellung. Das Business-Capability-Register mit Value Streams, Realisierungsbeziehungen und Panorama-Verknüpfung existiert. Worker Types können auf Operationsebene versionierte technische Capability-Claims deklarieren (CDA0). Die Engine enthält einen strikten semantischen Contract-Parser (CDA1), einen Offline-Prüfer für vollständige JSON-Schema-Dokumente, Content-Digests und Implementation-Digests. Seit dem Merge von PR #1401 gibt es zusätzlich einen Evidence-Record-Typ, einen reproduzierbaren Verifier und ein CI-Gate für die Konsistenz vorhandener Records. Dies ist die technische Grundlage von CDA2, aber **noch kein publizierter, unabhängig verifizierter Provider-Vertrag**. Die Verzeichnisse für publizierte Contracts und Evidence Records sind auf dem geprüften `main` nicht vorhanden. Im Programm und der Roadmap wird PR #1401 fälschlich noch als «in Review» geführt; der PR ist seit dem 27. Oktober 2026 gemerged. Programm · ADR-0483 · Evidence-Verifier.

Nicht geliefert sind der providerübergreifend verifizierte Operationenkatalog (CDA3), der technische Architekturgraph (CDA4), die darauf aufbauende Discovery (CDA5), eine capabilitybasierte BPMN-Deployment-Bindung (CDA6), der so gebundene, vollständig autorisierte Ausführungspfad (CDA7) und kontrolliertes Worker-Rebinding ohne erneutes BPMN-Deployment (CDA8). Vorhandene Worker-Jobs und ein teilweise realisierter delegierter SharePoint-Pfad sind **keine** Beweise für diese CDA-Funktionen. Roadmap · ADR-0482 · ADR-0475.

Die vorrangige nächste fachlich demonstrierbare Erweiterung ist eine veröffentlichte, präzise definierte Contract-Version mit unabhängig geprüfter Evidenz für **eine** konkrete Worker-Type-Operation. Erst danach sollten graphische Nachverfolgbarkeit, BPMN-Binding und Runtime-Routing als Funktionen beworben werden. **Einschätzung:** Das schützt vor dem zentralen Fehlversprechen, dass zwei ähnlich benannte API-Methoden schon austauschbar seien.

1. Ausgangslage und Problemstellung

Prozesse sind aus Sicht des Fachbereichs Mittel zur Erbringung einer Leistung. Eine Business Capability wie «Identität verwalten» formuliert eine Fähigkeit der Organisation unabhängig von einer konkreten SaaS-Plattform. Die technische Ausführung arbeitet hingegen mit spezifischen Ressourcen, Methoden, Berechtigungen und Seiteneffekten: ein Microsoft-Entra-Konto ist kein LDAP-Eintrag und kein Google-Kontakt. Ein unkontrollierter Übergang von solchen Bezeichnungen zu automatischem Service-Task-Routing erzeugt fachlich falsche Ergebnisse oder Sicherheitsrisiken.

Die schon implementierte atlas-Architektur trennt Business-Capability-Register, BPMN-Engine und Worker-Katalog. Die fehlende Verbindung ist nicht einfach ein weiteres Datenbank-JOIN. Es sind mehrere verschiedenartige Aussagen mit unterschiedlichen Beweisanforderungen: Eine Architektin *modelliert* die Realisierung einer Business Capability. Ein Worker-Type-Manifest *behauptet* einen technischen Vertrag. Eine unabhängige Prüfinstanz kann die Übereinstimmung *nachweisen*. Eine Policy erlaubt oder verweigert eine *konkrete externe Aktion*. Nicht jede dieser Aussagen entsteht zur gleichen Zeit, hat dieselbe Verantwortlichkeit oder darf dieselbe technische Wirkung haben.

Eine nachvollziehbare Automation muss ausserdem Aussagen zur Version, zum ausführenden Worker, zur Zielidentität, zur Autorisierung und zur Recovery machen. Andernfalls ist eine übersichtlich gezeichnete Capability-Landkarte noch kein belastbares Ausführungsmodell. CDA ist deshalb als eigenständiges Programm mit expliziten Gates und vertikalen Slices CDA0–CDA8 geführt. Programm.

2. Begriffe, Ebenen und Beweislast

Begriff	Bedeutung und notwendige Abgrenzung
Business Capability	Fachliche Fähigkeit einer Organisation; unabhängig von Provider, Schnittstelle oder Prozessmodell.
Value Stream / Stage	Wertschöpfungsfluss beziehungsweise Schritt mit Bezug auf benötigte Business Capabilities.
Realisierung	Von der Architektur fachlich erfasste Beziehung zwischen Capability und BPMN-Prozess; kein aus API-Aufrufen errechneter Beweis.
Technical Capability Contract	Versionierte, providerunabhängige semantische Beschreibung einer genau definierten Operation, einschliesslich Ressourcenart, Identitätsscope, Daten, Fehlern und Wirkungen.
Claim	Deklaration einer Worker-Type-Operation, einen bestimmten Contract zu unterstützen; bleibt ohne Nachweis eine Behauptung.
Evidence / Conformance	Nachvollziehbar prüfbare Evidenz einer konkreten Implementierung gegen einen bestimmten Contract und ein bestimmtes Testprofil.

Begriff	Bedeutung und notwendige Abgrenzung
Worker Type	Installierbare Implementierung beziehungsweise Integrationstyp mit Operationen und Job Types.
Worker	Konfigurierte Instanz eines Worker Types: Ziel, Einstellungen und referenzierte Zugangsdaten.
Worker Instance	Laufender, Jobs leasender Worker-Prozess. Ein Worker kann mehrere laufende Instanzen haben.
Binding	Eindeutige Zuordnung eines BPMN-Service-Tasks zu einer definierten Operation und einem konkret konfigurierten Worker.
Autorisierung	Entscheidung für die konkrete externe Aktion im gegebenen Identitäts-, Policy- und Ressourcenkontext; nicht aus dem Contract ableitbar.

Die Begriffe folgen ADR-0203, ADR-0480 und ADR-0485. Die technische Contract-Identität besteht aus `id`, exakter SemVer-`version` und optionalem `profile`. Das Profil ist **keine** stillschweigende Erweiterung auf alle Provider. Insbesondere gilt aktuell die Manifest-Regel: Pro Operation darf nur ein Claim zu derselben `id@version` existieren, auch bei verschiedenen Profilen. ADR-0483.

3. Architekturüberblick und Ist-Zustand

Der fachliche Teil basiert auf Design-Time-Records in `api/capability`: Business Capabilities, Value Streams und ihre Stufen, Realisierungen, Coverage und Gap-/Messperspektiven. Panorama kann das Capability-Register über seine Architekturansichten referenzieren. Dies ist kein neuer Laufzeitspeicher und kein technischer Capability-Graph. ADR-0305 ist **Accepted / Partial**; ADR-0308 ist **Accepted / Landed**. Die Quellmodule und zugehörigen Tests liegen unter `api/capability` und `api/panorama`.

Der Runtime-Kern bleibt die eventbasierte BPMN-Engine: Deployment kompiliert ein Modell zu einem unveränderlichen, integer-indexierten Graphen. Ein Single-Writer-Processor je Partition verarbeitet Commands, erzeugt Events und macht diese vor externen Effekten im WAL dauerhaft. State ist eine Materialisierung der Events; Live-Verarbeitung und Recovery verwenden denselben deterministischen `applyToState`-Pfad. Ein Service Task kann einen dauerhaften Job erzeugen, den eine Worker Instance nach dem Commit leased. Diese vorhandene Engine-Semantik wird durch CDA0–CDA2 nicht verändert. Architektur · Invarianten.

Die Worker-Type-Metadaten sind die heutige Erweiterungsstelle. Das vorhandene Manifest führt Operationen und optionale Capability-Claims; ein eigenständiger Contract-Parser prüft semantische Dokumente ausserhalb des Processor-Hot-Paths. Ein existierender Worker-Katalog ist deshalb nicht mit einem öffentlich nutzbaren, verifizierten Capability-Katalog gleichzusetzen. Ebenso wenig ist das spätere ADR-0420-Graphmodell aus den heutigen Panorama-Ansichten ableitbar: ADR-0420 steht auf **Proposed / Not started**.

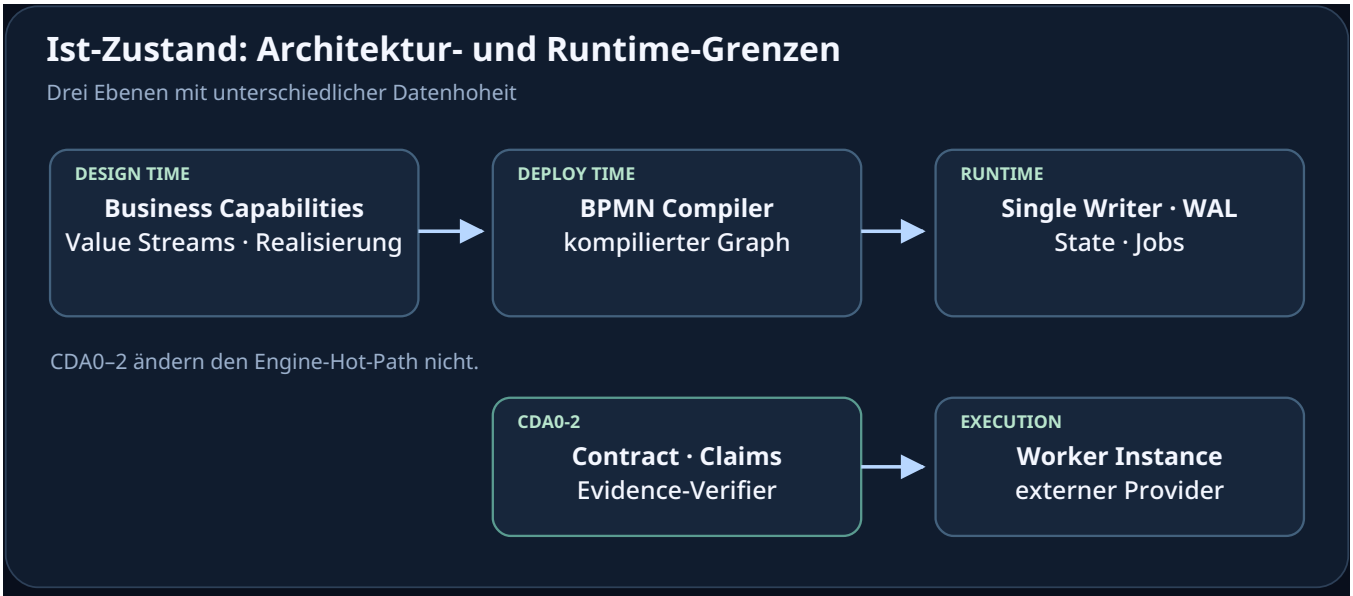


Abbildung: Architekturgrenzen und Einordnung des implementierten Bestands.

4. Von der Facharchitektur zum technischen Vertrag

Das Zielbild beginnt mit einem Value Stream, dessen Stage eine Business Capability benötigt. Ein Architekt ordnet einen BPMN-Prozess dieser Fähigkeit als *Realisierung* zu. Ein Service Task kann im späteren CDA6-Modell eine explizite technische Operation referenzieren. Die Operation erhält ihren präzisen Vertrag nicht aus ihrem Anzeigenamen, sondern aus einer getrennt versionierten, konkreten Contract-ID. Über den Claim wird zunächst nur ausgedrückt, dass eine Worker-Type-Operation diese Semantik anbietet.

Die Verbindung ist nicht automatisch transitiv. Aus «Prozess führt Entra-Operation aus» folgt nicht «Prozess realisiert fachlich Identitätsmanagement». Die fachliche Realisierung bleibt ein bewusst erfasstes Architektur-Faktum. Aus «Worker Type behauptet Contract» folgt nicht «Provider verhält sich identisch». Erst überprüfte Evidenz kann einen solchen Schluss in einer definierten, begrenzten Form tragen. Und selbst dann erlaubt ein technischer Contract nicht automatisch die konkrete Aktion. Der konzeptionelle Datenfluss lautet somit: *authored → claimed → verified → bound → authorized → observed*. Die Ausdrücke bezeichnen verschiedene Vertrauens- und Verantwortungsgrenzen. ADR-0480.

Das versprochene Nutzenbild umfasst Impact-Abfragen («Welche Value Streams hängen von diesem Worker ab?»), Gap-Abfragen («Wo fehlt ein verifizierter Implementierungspfad?») und später kontrolliertes, autorisiertes Routing. Diese Abfragen benötigen jedoch eindeutige, nachvollziehbare Kanten und explizite Sichtbarkeitsregeln. Die derzeitige Read-Only-Architektur für CDA4/CDA5 ist geplant; sie darf in einer Präsentation nicht als vorhandenes Feature dargestellt werden.



Abbildung: Unterschiedliche fachliche, deklarative und geprüfte Beziehungen.

5. CDA0: deklarative Claims – umgesetzt

CDA0 ist im definierten Umfang abgeschlossen. Die Manifest-Struktur in `workertype/capability.go` enthält `CapabilityClaim` mit `id`, `version` und optionalem `profile`. Der strikte Manifest-Parser validiert neue und alte Beschreibungen; der bisherige Einsatz ohne Claims bleibt kompatibel. Die Änderungen wurden mit PR #1343 gemerged; ADR-0480 ist akzeptiert, gesamthaft jedoch nur teilweise implementiert.

Ein Claim beschreibt *eine Operation* und nicht eine unspezifische Leistungsfähigkeit des gesamten Worker Types. Ausserdem ist ein Claim kein Protokoll zur Selbstfreigabe. Er erteilt keine Runtime-Rechte, wählt keinen Worker und garantiert weder gleiches Fehlerverhalten noch gleiche Ressourcenidentität. Genau diese Zurückhaltung ist ein Architekturmerkmal: Ein Manifest kann bereits maschinenlesbare semantische Aussagen enthalten, ohne dass die Engine sie ungeprüft in automatische Aktionen übersetzt.

Die Manifest-Ebene muss zudem von der Worker Type-Verteilung unterschieden werden. ADR-0208 ist **Accepted / Partial**: das Paket- und Vertrauensmodell ist nicht in allen vorgesehenen Aspekten fertig. Die Existenz von Capability-Claims ersetzt keine Prüfung von Signaturen, Herkunft oder unveränderlicher Paketidentität. **Einschätzung:** Für eine sicherheitskritische Umgebung ist dies ein wichtiger Punkt, weil ein technisch passender, aber unzureichend vertrauenswürdiger Worker keinesfalls allein aufgrund eines Claims zulässig werden darf.

6. CDA1: semantischer Contract-Envelope – umgesetzt

Der mit PR #1346 gelieferte Contract-Envelope ist eine eigenständige, streng geparste Beschreibung. Der öffentliche Go-Typ unter `workertype/contract.go` verwendet die API-Version `atlas.io/technical-capability/v1alpha1` und den Kind-Wert `TechnicalCapabilityContract`. Der Vertrag umfasst unter anderem die semantische Contract-Identität, Ressourcenart, Identitätsscope und Input-/Output-Schema-Wurzeln. Die Prüfung erfolgt offline beziehungsweise zur Design-Time, nicht pro Job.

Die Modellierung verlangt explizite Aussagen über Abwesenheit und `null`, Fehlerkategorien, Idempotenz, partiellen Erfolg, Konsistenz und Seiteneffekte. Der Contract ist semantisch immutable: Ändert sich seine Bedeutung, wird eine neue Contract-Version benötigt. Die Annahme eines exakt versionierten Dokuments verhindert nicht alle semantischen Missverständnisse; sie macht aber ein überprüfbares Ziel für Conformance-Tests möglich. Die **v1alpha1-Felder** dürfen gemäss ADR-0485 bis zur ersten Veröffentlichung noch angepasst werden, während Contract-Identität und Immutabilität verbindlich bleiben.

Das ist mehr als ein Namensregister, aber weniger als eine nachgewiesene Fähigkeit. Ein formal valides Schema kann ein unzutreffendes Verhalten einer Entra-Operation nicht erkennen, solange keine realistische, geprüfte Erwartung und keine Ausführungs-Evidenz gegenüberstehen. Daraus folgt der eigene Ausbauschritt CDA2.

7. CDA2: Governance, Conformance und Evidenz – teilweise umgesetzt

Der Stand von CDA2 muss präzise nach Bausteinen ausgewiesen werden:

Baustein	Geprüfter Stand auf `main`	Einschränkung
Vollständige Draft-2020-12-Schema-Prüfung	**Umgesetzt**: `workertype/conformance.go`, PR #1357	Offline-Prüfung von Beispielen; kein Nachweis einer echten Provider-Implementierung.
Contract-Content-Digest	**Umgesetzt**: `workertype/contract_digest.go`, PR #1360	SHA-256 über exakte LF-Dateibytes; veränderte Bytes ergeben einen anderen Digest.
Implementation-Digest	**Umgesetzt**: `internal/impldigest`, PR #1388	Bindet eingebaute Worker-Implementierungspfade; kein allgemeines Drittanbieter-Vertrauenssiegel.
Evidence-Record-Typ und Verifier	**Umgesetzt als Infrastruktur**: `workertype/evidence`, PR #1401	Strikt gebundene, reproduzierbar prüfbare Records; kein publizierter Record vorhanden.
Automatisches Gate	**Umgesetzt als Mechanismus**: `go test ./workertype/evidence/gate`	Verifiziert vorhandene Records; bei leerer Sammlung ergibt sich daraus keine Provider-Konformität.

Baustein	Geprüfter Stand auf `main`	Einschränkung
Publizierte Contract-/Evidence-Artefakte	**Geplant/offen**	`workertype/contracts/` und `workertype/evidence/records/` fehlen im geprüften `main`.
Withdrawal und öffentlich abfragbarer Claim-Status	**Geplant/offen**	Keine implementierte Withdrawal-/Claim-Statusprojektion nachgewiesen.

Die gesonderten Evidence Records bilden eine konkrete Bindung aus Contract, Worker Type, Operation und Suite. Der Verifier leitet dazu Identität und Digests aus dem Modul sowie aus den Implementierungspfaden ab. Ein Record muss beim erneuten Ableiten bytegleich entstehen oder als veraltet beziehungsweise unzulässig erkannt werden. Eine Suite führt festgelegte Fälle gegen *Fakes* aus; sie kennzeichnet Ergebnisse als `passed`, `failed` oder `not-exercised`. Nicht ausübbares reales Provider-Verhalten darf nicht als bestanden ausgegeben werden. Der Approval-Nachweis stammt aus dem Repository-Review und nicht aus einem frei gesetzten JSON-String. ADR-0483 · `evidence.go` · `suite.go`.

Statuskorrektur gegenüber der Roadmap: PR #1401 wurde am **27. Oktober 2026** gemerged, obwohl `ROADMAP.md` und das CDA-Programm ihn noch als «in Review» bezeichnen. Die implementierten Record-/Gate-Bausteine sind deshalb in diesem Whitepaper auf «Umgesetzt als Infrastruktur» angehoben. CDA2 bleibt dennoch «Teilweise umgesetzt», weil die erste veröffentlichte Contract/Evidence-Kombination, eine geprüfte Provider-Operation und Withdrawal fehlen. Ein leerer Bestand wird nicht zu einem bestandenen Konformitätsnachweis umgedeutet.

Sichere Implementierung bedeutet ausserdem, dass der gesamte überprüfte Runtime-Code einem Implementation-Digest zugeordnet ist. Die in ADR-0483 dokumentierte erste Governance setzt auf maintainergeprüfte Artefakte und eine einzelne Approval-Autorität; externe Signaturen, Drittanbieter-Attestierungen und umfassende Live-Tenant-Tests sind bewusst noch keine generellen Zusagen. Der Conformance-Runner erzeugt keine Runtime-Berechtigung.

8. CDA3: verifizierte, aus der Quelle abgeleitete Worker-Claims – geplant

CDA3 soll die konkrete Worker-Type-Operation mit einem präzisen Contract verbinden, ohne eine parallele Operationstabelle aufzubauen. Die Quelle einer Operation ist die vom Worker Type tatsächlich angebotene Operation samt Job Type. Ein Read-only-Katalog darf daraus Metadaten ableiten; er darf aber eine bequeme Mapping-Tabelle nicht als zweite Wahrheit etablieren. Für jede Zuordnung sind Contract-Identität, Profil, Version, Evidenzquelle und gegebenenfalls eine explizite Ablehnung der Gleichwertigkeit erforderlich. Programm · ADR-0480.

Die Negative Cases sind mindestens so wichtig wie der Erfolgspfad: Eine Google-Contacts-Operation, die einen Kontakt erzeugt, darf niemals allein wegen des Verbs «create» als Entra-Account-Erstellung gelten. Eine LDAP-Operation kann einen Verzeichniseintrag erzeugen, ohne dass er die Identitätsscope- und Lifecycle-Semantik eines Benutzerkontos erfüllt. Selbst innerhalb eines Providers können Fehlercodes, Retry-Fähigkeit, Pagination oder Zeitkonsistenz abweichen. Erst ein konkreter Contract plus ein passender Evidenzumfang erlaubt die Aussage

«verified»; andernfalls sind «candidate», «unknown» oder «unsupported» die einzig ehrlichen Zustände.

CDA3 ist nach Programm und Codeprüfung noch nicht geliefert. Der vorhandene Mechanismus der Evidence Records ist eine Voraussetzung, aber sein Vorhandensein erzeugt keine Claims für Entra, LDAP oder Google Contacts. **Einschätzung:** Beginnen sollte man mit einer eng umrissenen Account- oder Contact-Operation und einer Matrix dokumentierter Nicht-Äquivalenzen. Das schafft ein ausführbares Qualitäts-Gate, bevor ein grösserer Katalog vermeintlich gleichwertiger Fähigkeiten entsteht.

9. CDA4 und CDA5: Architekturgraph, Discovery und Impact – geplant

Ein Modell, mehrere Projektionen

Der geplante technische Erweiterungsgraph verbindet die bereits vorhandenen Business-Architecture-Datensätze mit Prozessdefinitionen, explizit gebundenen Service Tasks, Operationen, Contracts und konkreten Workern. Er ist keine zusätzliche, transaktional gepflegte Wahrheit über dieselben Objekte. Beziehungen müssen nach Quelle und Bedeutung unterscheidbar bleiben: *authored* für fachlich erfasste Realisierungen, *derived* für deterministisch auflösbare Verweise, *claimed* für Manifest-Aussagen, *verified* für unabhängige Evidenz und *observed* für tatsächlich ausgeführte Jobs und Aktionen. ADR-0420 ist **Proposed / Not started** und liefert somit kein heute benutzbares Graph-API.

Eine robuste Impact-Antwort muss ferner nachweisen, welche Kante auf welches Artefakt und auf welche Version zurückgeht. Wird ein Worker ausgetauscht, darf eine alte Evidenz nicht stillschweigend auf den neuen Code vererbt werden. Werden BPMN-Definitionen neu deployed, muss die Abfrage die verwendete Definition und Version kennen. Für vertrauliche Bereiche gilt «stop at restricted»: nicht nur Knotennamen, sondern auch vermeintlich harmlose Anzahlen oder indirekte Pfade können Informationen offenlegen. Die tatsächliche Berechtigung ist für jede Projektion zu prüfen.

Was Discovery liefern soll

Geplante CDA5-Abfragen umfassen: Welcher Value Stream hängt von einer bestimmten Operation ab? Welche Prozesse adressieren einen Contract, für den keine verifizierte Implementierung vorhanden ist? Welche konfigurierte Worker-Instanz wäre von einem sicherheitsbedingten Withdraw betroffen? Wo besteht eine Lücke zwischen Business Capability und technisch belegter Umsetzung? Solche Antworten müssen unvollständige Daten als unvollständig markieren, statt fehlende Kanten stillschweigend zu ergänzen.

Heute existieren Capability-Register, Panorama-Bindings, einige Coverage-/Gap-Funktionen und Worker-Übersichten. Das ist fachlich wertvoll, aber **noch keine CDA5-End-to-End-Discovery**. Der Weg von Business Capability zu einem unabhängig verifizierten Technical Contract ist noch nicht implementiert; erst recht kann daraus keine dynamische Routenberechtigung folgen. CDA5

ist keine technische Voraussetzung für jeden frühen CDA6-Prototyp, wohl aber für eine nachvollziehbare Benutzererfahrung über mehrere Architektur-Ebenen. Programm.

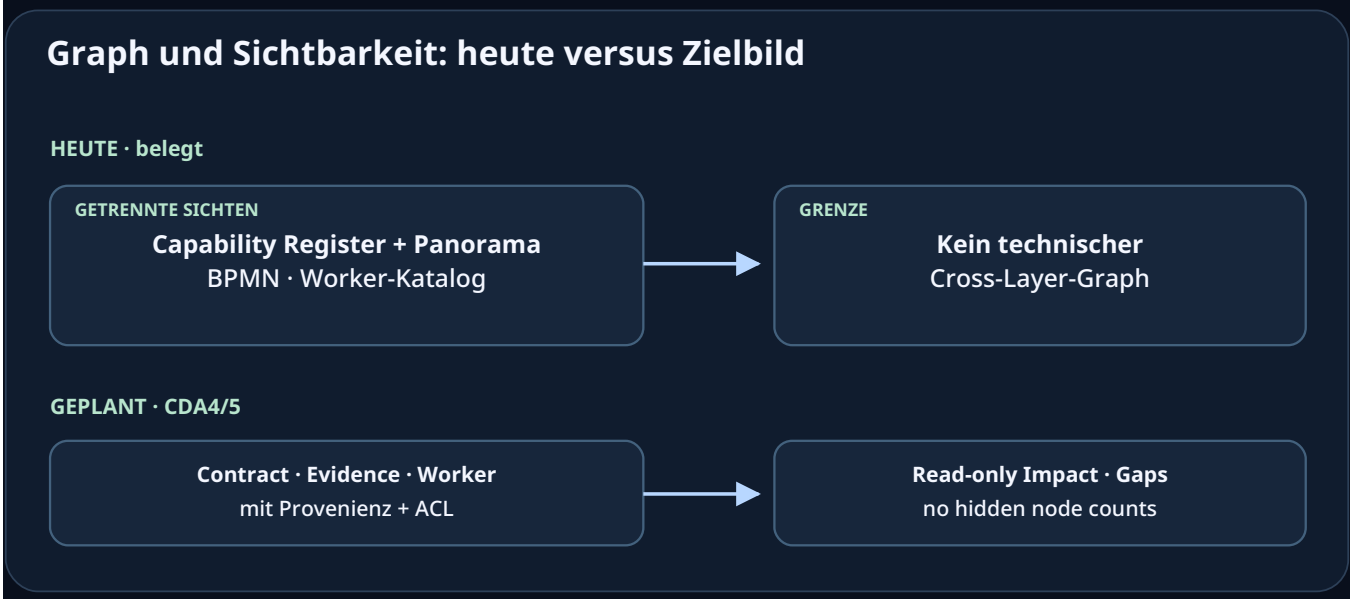


Abbildung: Bestehende Daten und geplante autorisierte Cross-Layer-Abfragen.

10. CDA6: BPMN-Binding und Deployment-Validierung – geplant

Im ersten geplanten Modell erklärt ein Service Task **opt-in** den exakten technischen Contract und wählt einen passenden, überprüften konfigurierten Worker. Beim Deployment werden Contract-Identität, Profil, Digest, Worker-Type-Version und konkrete Worker-Zuordnung validiert; eine fehlende, unklare oder nicht mehr gültige Bindung führt zu einer verständlichen Fehlermeldung. Die bestehende BPMN-Ausführung ohne dieses neue Feld bleibt gültig. Weder BPMN `Task` noch `Call Activity` wird semantisch uminterpretiert. ADR-0482.

Die Kompilierungsinvariante ist entscheidend: XML, semantische Vertragssuche, String-Auflösung und statische Prüfung gehören in den Deployment-Pfad. Zur Laufzeit wird ein bereits festgelegter Plan verwendet. Ein unveränderlicher BPMN-Deployment-Vertrag darf nicht unbemerkt durch veränderte Worker-Kataloge umgeschrieben werden. Das geplante Deployment-Ergebnis muss deshalb eindeutig sein und unabhängig von späteren Katalogänderungen bleiben.

CDA6 soll allerdings schon bei der Gestaltung die mögliche spätere Umschaltung berücksichtigen: Ein stabiler **logischer Binding-Slot** dient als Identität der Bindungsstelle, auch wenn zunächst eine konkrete Worker-Revision fest gepinnt bleibt. Damit kann CDA8 künftig eine neue freigegebene Zielrevision einführen, ohne die BPMN-Definition selbst zu verändern. Dieser Entwurfszwang ist **keine** vorweggenommene Runtime-Rebinding-Funktion. Die XML-Syntax, die Diagnostik und das Authz-Modell benötigen noch die im ADR beschriebenen Architekturentscheidungen.

11. CDA7: Governed Execution – geplant

Im ersten ausführbaren CDA-Slice soll die Engine die beim Deployment eindeutig gewählte Bindung nutzen und einen Job für genau diesen, bereits überprüften Worker bereitstellen. Vor der externen Aktion muss die unterstützte Identity-Aware-Execution-Schnittstelle eine konkrete, autorisierte Entscheidung für Ziel, Identität, Operation und Ressource liefern. Verweigerung bedeutet Verweigerung: kein versteckter Fallback auf einen anderen Worker. Eine positive Contract-Konformität ist dafür weder Berechtigung noch eine Security Policy.

Das Programm IAE ist hier eine eigenständige Abhängigkeit. Seine ADR-0475 ist **Proposed / Partial**; dokumentiert sind ein begrenzter, beaufsichtigter Microsoft-SharePoint-Pfad und offene Provider- und Governance-Fragen. Daraus folgt keine bereits vollständige Freigabe für Entra, LDAP, Google Contacts oder einen beliebigen Drittanbieter-Worker. CDA7 kann erst für einen konkreten Provider gelten, wenn dessen Policy Enforcement tatsächlich im Code, in Tests und auf der richtigen Betriebsgrenze nachgewiesen ist.

Recovery ist Bestandteil der fachlichen Garantie

Ein Service-Task-Job ist keine einmalige HTTP-Anfrage, die im Erfolgsfall vergessen wird. Die Engine persistiert Zustandsänderungen als Events, versieht Jobs mit Lease-/Fencing-Semantik und kann sie nach einem Neustart wiederherstellen. Eine spätere Capability-Bindung darf diese Eigenschaften nicht abschwächen. Insbesondere muss die zum Zeitpunkt der Ausführung gewählte, autorisierte Zuordnung als dauerhafte Tatsache festgehalten sein, bevor sie extern sichtbar wird. Beim Replay darf weder ein neuer Worker gewählt noch eine aktuelle externe Policy oder ein Live-Provider befragt werden. Die neue Autorisierung gehört in die dafür akzeptierte Ereignis-/Ausführungsarchitektur, nicht in `applyToState`.

Die unverhandelbare Reihenfolge ist: Command bearbeiten; Events erzeugen; WAL schreiben; gemeinsam `fsync` ausführen; State committen; Followups bereitstellen; erst dann Responses, Notifications oder externe Side Effects ausführen. Der Single Writer pro Partition bleibt alleiniger Besitzer seines State. Eine neue Binding-Funktion darf weder Locks im regulären Prozesszustand noch Heap-Allokationen pro Command im Hot Path oder direkte Cross-Partition-Zugriffe einführen. Invarianten.

Wichtig zur Zustellsemantik: «durable before visible» schützt den eigenen Engine-Zustand vor voreiliger Sichtbarkeit. Es liefert nicht automatisch eine exakt-einmalige externe API-Aktion. Worker verarbeiten geleaste Jobs unter Retry- und Failure-Semantik; externe Aktionen benötigen geeignete Idempotenz, Schlüssel und Reconciliation. Eine Recovery, die die interne Eventhistorie korrekt rekonstruiert, kann die bereits ausgeführte Fremdaktion nicht einfach zurückrollen. Diese Grenze ist in jedem späteren Runtime-Whitepaper und jedem Betriebsvertrag transparent zu machen. ADR-0203 · Architektur.

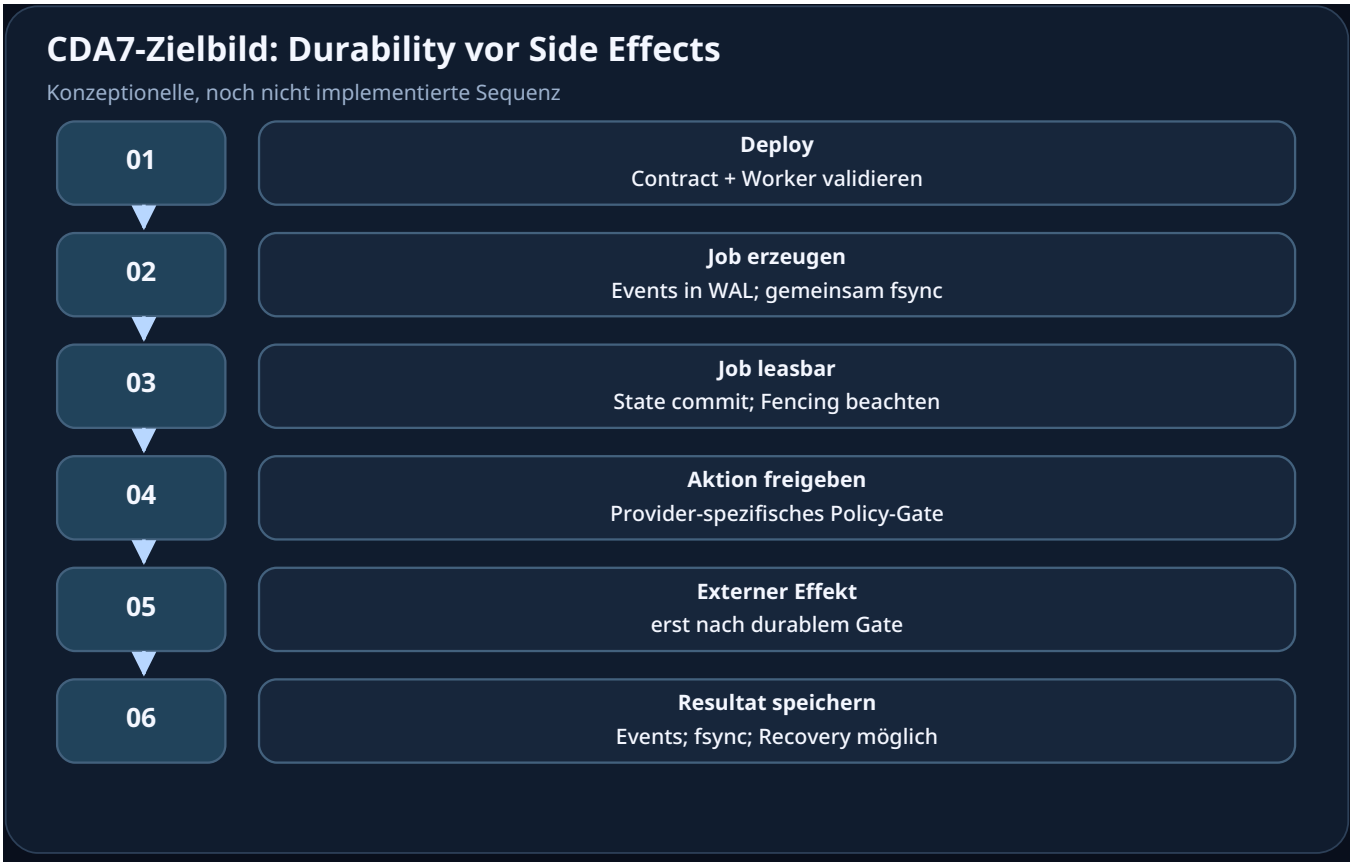


Abbildung: Konzeptionelle Abfolge für einen späteren CDA7-Ausführungspfad.

12. CDA8: kontrolliertes Rebinding – geplant, Entscheidung zurückgestellt

Das weitere Ziel ist der explizit genehmigte Austausch von **konfiguriertem Worker A** zu **Worker B** für zukünftige Jobs **einer bestehenden, opt-in bereitgestellten BPMN-Prozessversion**, ohne die Prozessdefinition erneut zu deployen. Der Contract bleibt semantisch identisch; die Bindung erhält eine neue Revision. Ein Operator muss eine Vorschau über Übereinstimmung, Berechtigung, betroffene Instanzen und offene Jobs erhalten. Eine leise Automatik ist ausdrücklich nicht vorgesehen. ADR-0492.

Die notwendige zeitliche Trennung lautet: Jobs, die vor dem dauerhaft bestätigten Cutover entstanden sind, behalten ihre ursprüngliche Zielrevision. Ein laufender oder geparkter Job darf nicht von selbst zu Worker B wandern. Für neue Jobs wird die neue Revision erst nach einem für die zuständige Partition dauerhaft gespeicherten Entscheid sichtbar. Bei mehreren Partitionen kann ein Rollout deshalb vorübergehend teilweise abgeschlossen sein; diese Differenz muss sichtbar und betreibbar sein. Ein Rollback ist eine neue, autorisierte Revision und keine nachträgliche Umschreibung der alten Facts.

CDA8 ist **nicht implementiert** und seine Architektur ist **nicht angenommen**. Die Review-Fragen von ADR-0492 wurden laut ADR bis nach CDA7 zurückgestellt; lediglich die Frage nach dem Binding-Slot kann bei CDA6 vorgezogen werden. Eine automatische Worker-Auswahl pro Job wäre ein anderer Funktionsumfang mit eigenem ADR, eigener Event-Semantik,

Sicherheitsprüfung und Recovery. Daraus folgt: Selbst ein später erfolgreiches CDA7 darf nicht als «selbstheilendes providerneutrales Routing» beschrieben werden.

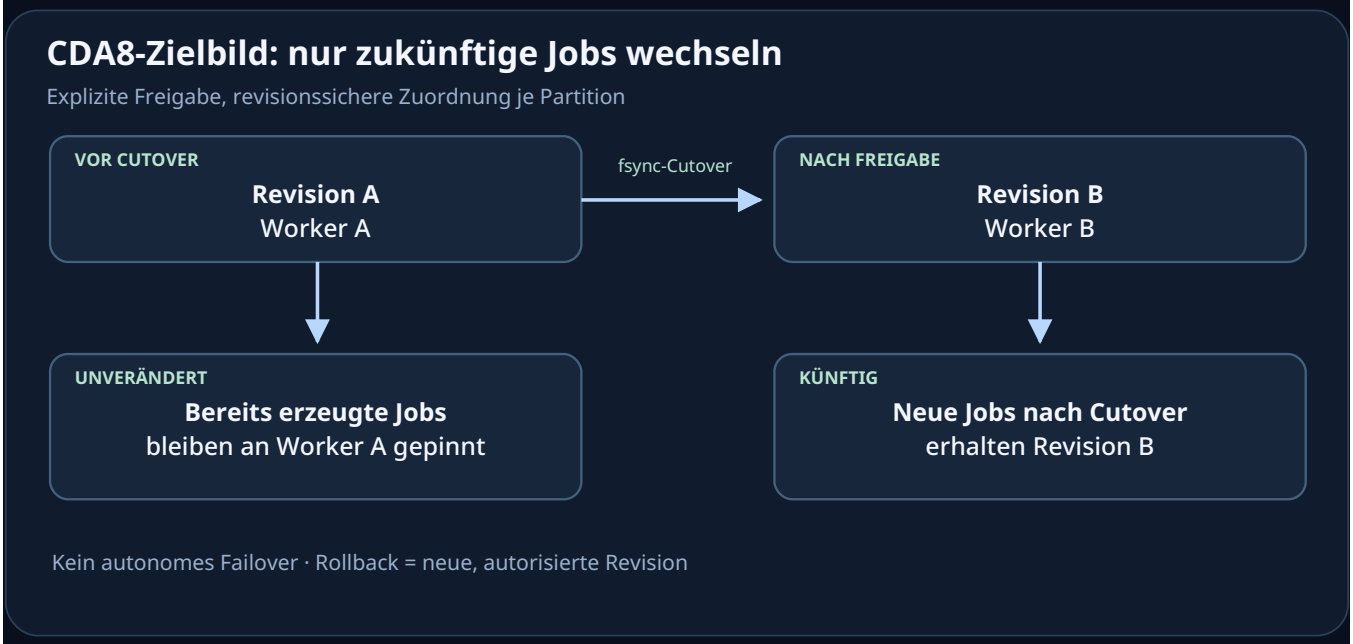


Abbildung: Trennung von bestehenden und neuen Job-Bindungen.

13. Sicherheit, Governance und Datenschutz

Die wichtigsten Schutzgüter sind: Integrität der fachlichen Zuordnungen, Semantik und Version der Verträge, Herkunft und Implementierung der Worker, Sichtbarkeit von Architekturbeziehungen, Identitäts- und Credential-Grenzen, korrekte Autorisierungsentscheidungen sowie dauerhaft nachvollziehbare externe Effekte. Jede Ebene hat andere Verantwortliche. Die Facharchitektur bestätigt die Realisierung; ein Contract-Maintainer verantwortet die Semantik; ein unabhängiger Reviewer bestätigt definierte Prüfergebnisse; der Betreiber entscheidet über Worker-Konfigurationen und Trust Roots; die Policy Engine entscheidet zur konkreten Aktion.

Bedrohung oder Fehler	Gegenmassnahme im Konzept	Status / Restrisiko
Falsche Gleichsetzung ähnlich benannter Provider-Operationen	Exakte Contract-Version, Profil und Negativ-Evidenz	Contract-Envelope umgesetzt; echte Provider-Verifikation offen.
Selbstzertifizierung eines Worker Types	Unabhängiges, digestgebundenes Evidence Review	Evidence-Infrastruktur vorhanden; keine publizierten Records.
Nachträgliche Änderung geprüfter Implementierung	Content-, Implementation- und Suite-Digests; CI-Gate	Teile umgesetzt; Revocation und publizierte Provenienz offen.
Fehlendes Ziel- oder Ressourcenrecht	Provider-spezifischer Policy Enforcement Point mit Identitätskontext	IAE teilweise; kein universelles Worker-Enforcement.
Informationsleck über Architekturgraph	Autorisierte, begrenzte Traversierung, keine versteckten Knotenzahlen	CDA4/CDA5 geplant.

Bedrohung oder Fehler	Gegenmassnahme im Konzept	Status / Restrisiko
Inkonsistenter Job-Routing-Entscheid beim Recovery	Durable Event vor Side Effect; keine Neuauswertung beim Replay	Engine-Invarianten vorhanden; CDA-Routing noch nicht implementiert.
Secrets oder persönliche Daten in Contract/Graph/WAL	Referenzen statt Tokenwerten, datensparsame Evidence und Zugriffskontrolle	Architekturvorgabe; bei neuen CDA-Funktionen noch nachzuweisen.

Die Governance in ADR-0483 verwendet anfangs eine einzelne Maintainer-Autorität mit Repository-Provenienz, nicht bereits ein föderiertes Zertifizierungs-Ökosystem. Die Trennung von Contract- und Implementation-Digest verhindert, dass eine Änderung der Implementierung automatisch einen unveränderten Conformance-Nachweis behält. Sie verhindert aber nicht sämtliche Risiken kompromittierter Prüfumgebungen, unvollständiger Testfälle, unechter Fakes oder falsch akzeptierter Semantik. Solche Fälle benötigen Review-Regeln, Negativtests und gegebenenfalls Live-Tenant-Gates.

Für sensible Bereiche gelten mehrere explizite Verbote: keine Credentials in BPMN, Capability-Metadaten oder Conformance-Records; keine geheimen Policy-Attribute in frei abfragbaren Graphantworten; keine unautorisierten Traversals über geschützte Architekturgrenzen. Die spätere Identity-Aware Execution benötigt nachvollziehbare Entscheidungen und Schutz gegen Tokenmissbrauch und Revocation-Races. Wo diese Pfade nicht nachgewiesen sind, ist dies als **offen** auszuweisen und nicht mit der allgemeinen Sicherheit der Event-Sourcing-Engine gleichzusetzen. IAE-Programm.

14. Durchgehendes Beispiel: Employee Onboarding

Ein Onboarding-Value-Stream umfasst beispielsweise die Stage «IT-Zugang bereitstellen». Der Fachbereich modelliert die benötigte Business Capability «Identity Management». Ein BPMN-Prozess wird als fachliche Realisierung zugeordnet und enthält einen Service Task «Konto im Zielsystem anlegen». Ein konkreter Entra Worker Type kann eine Operation zur Kontoanlage anbieten; deren technische Semantik muss einen Vertrag mit Ressourcenart *Account* und geeignetem Identitätsscope benennen. Der Worker ist mit Tenant, Zielumgebung und Vault-Referenzen konfiguriert. Eine spätere Policy entscheidet, ob diese Identität für diese Ressource diese Aktion ausführen darf.

Was bereits als Baustein gezeigt werden kann: Business-Capability-/Value-Stream-Modellierung, BPMN-Modell und -Ausführung, Worker-Type-Operationen, optionale Capability-Claims, der unabhängige Contract-Envelope, Schema- und Digest-Prüfungen sowie das Evidence-Gate als Infrastruktur. Für das gesamte Beispiel gibt es nach dem geprüften Stand jedoch keine veröffentlichte, verifizierte Entra-Account-Contract-Bindung. Es ist daher ausdrücklich **kein heute funktionsfähiger CDA7-End-to-End-Demoprozess**. ADR-0305 · ADR-0483.

Zielbild nach CDA3–CDA7: Ein unabhängig geprüftes Account-Contract-Artefakt und der dazugehörige Evidence Record erlauben die Kennzeichnung der exakt passenden Operation. CDA4/CDA5 zeigen die verantwortete Kette für Architektur- und Betriebssicht. CDA6 validiert eine

eindeutige initiale Bindung beim Deployment. CDA7 führt den Job ausschliesslich über den autorisierten Pfad aus und bewahrt Entscheidung und Ergebnis replaybar. Jede dieser Etappen hat ein eigenes Gate; ihr Erfolg darf nicht aus dem Vorhandensein des vorherigen Bausteins extrapoliert werden.

Negativ- und Ausfallfälle

1. **Google-Kontakt statt Entra-Konto:** gleiche Aktion «erstellen», aber andere Ressource und Identitätssemantik. Kein Contract-Match ohne belegtes Profil.
2. **Veralteter Evidence-Digest:** Quellcode oder Suite hat sich geändert. Die Prüfung muss den Record als stale erkennen; ein bisheriger Claim darf daraus nicht stillschweigend «verified» bleiben.
3. **Kein Zugriff auf den Tenant:** Ein korrektes Binding kann die Autorisierung nicht ersetzen; die externe Aktion wird verweigert. Kein transparenter Wechsel auf einen anderen Worker.
4. **Geschützte Abteilung:** Eine Nutzerin ohne Berechtigung darf nicht einmal aus einer Impact-Antwort auf die Anzahl versteckter Worker schliessen.
5. **Neustart nach dauerhafter Bindungsentscheidung:** Replay übernimmt gespeicherte Fakten; es darf weder einen neuen Worker ermitteln noch erneut eine externe Kontoerstellung auslösen. Externe Retry-/Idempotenz-Semantik muss separat beachtet werden.
6. **Worker A fällt nach einem künftigen Rebinding aus:** Alte, bereits A zugeordnete Jobs bleiben nicht automatisch auf B lauffähig. Der Operator braucht eine sichtbare Drain-/Incident-/Reconciliation-Regel.

Die ersten beiden Fälle prüfen Semantik und Evidenz, der dritte Sicherheitsgrenzen, der vierte Datenschutz, der fünfte die Engine-Invarianten und der sechste die zeitliche Bindungssemantik. Diese Trennung verhindert, dass ein einzelner «Happy Path» eine nicht gegebene Gesamtgarantie suggeriert.

15. Programmstatus CDA0–CDA8 und Abhängigkeiten

Slice	Am geprüften Commit	Nächster belastbarer Nachweis	Architektur-Gate
CDA0 Claims	**Umgesetzt** für deklarative Operation-Claims	Legacy-/Negativtests und dokumentierte Semantik behalten	ADR-0480 Accepted / Partial
CDA1 Contract-Envelope	**Umgesetzt** als Format/Parser	Ersten unveränderlichen Contract veröffentlichen	ADR-0485 Accepted / Partial
CDA2 Conformance	**Teilweise umgesetzt**; Validator, Digests, Evidence-Record/CI-Gate gebaut	Veröffentlichtes Contract-Artefakt, nachvollziehbar geprüftes Provider-Evidence, Withdrawal	ADR-0483 Accepted / Partial

Slice	Am geprüften Commit	Nächster belastbarer Nachweis	Architektur-Gate
CDA3 Worker-Mappings	**Geplant/offen**	Eine nachweislich richtige, aus Quelle abgeleitete Provider-Operation und Negativ-Mappings	CDA2 und ADR-0208-Trust-Gates
CDA4 Architekturgraph	**Geplant/offen**	Provenienz- und ACL-sichere technische Kanten, stabile IDs, Query-Grenzen	ADR-0420 aktuell Proposed
CDA5 Discovery	**Geplant/offen**	Read-only Impact-/Gap-Abfrage ohne verdeckte Knoten-Leaks	CDA4; UI verspricht keine Runtime
CDA6 BPMN-Binding	**Geplant/offen**	Verifizierter, eindeutiger Deployment-Plan; stable binding slot	ADR-0482 aktuell Proposed; CDA2/CDA3
CDA7 Governed Execution	**Geplant/offen**	Ein korrekt autorisierter und recoverygetesteter Provider-Pfad	CDA6 und konkretes IAE-Enforcement
CDA8 Controlled Rebinding	**Geplant/offen**	Expliziter Cutover mit alten Jobs, Rollback, Partition-Recovery	ADR-0492 Proposed; Review zurückgestellt

Die ursprüngliche T0–T5-Planung wurde in CDA0–CDA8 fachlich feiner aufgeteilt: T0 → CDA0; T1a → CDA1; T1b → CDA2; T2 → CDA3; T3 → CDA4; T4 → CDA5; T5 → CDA6 plus CDA7; T5c → CDA8. Dabei ist eine read-only Discovery kein zwingendes Engine-Dependency-Gate für jeden Validierungsprototyp. Vor einer fertigen produktiven Nutzersicht muss die Herkunft der Daten jedoch nachvollziehbar sein. Technischer Plan · CDA-Programm.

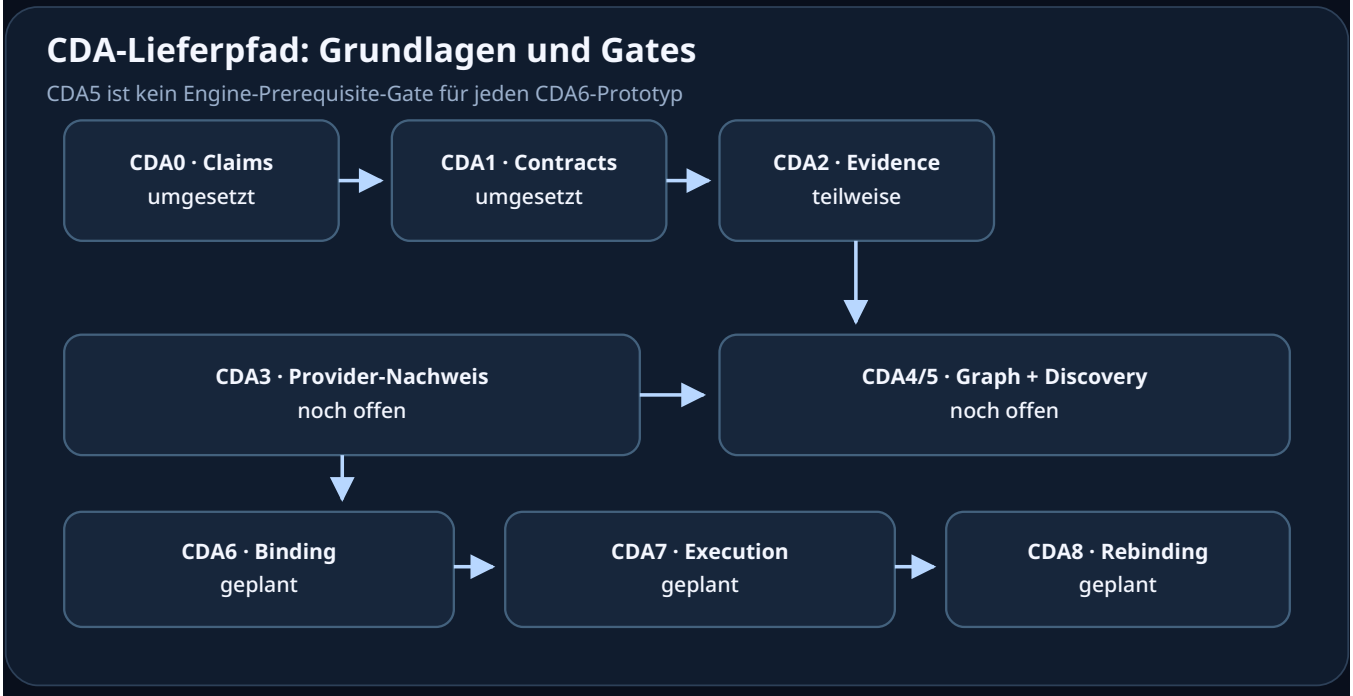


Abbildung: CDA-Slices und ihre wesentlichen Abhängigkeiten.

16. Alternativen, Grenzen und Trade-offs

Direkte Provider-Kopplung ohne CDA. Sie hat die geringste konzeptionelle Komplexität und ist für wenige kontrollierte Zielsysteme leicht verständlich. Die fachliche Abhängigkeit vom Provider wird jedoch oft in BPMN, Skripten oder Workerkonfiguration versteckt; Auswirkungen auf Value Streams, Versionswechsel und Beweisqualität bleiben manuell. Nicht jede Integration braucht deshalb sofort CDA: Für ein kleines, festes Prozessportfolio kann ein expliziter Worker Type ausreichen.

Manuell gepflegter Architektur- und Contract-Katalog. Er kann schnell Übersicht liefern, birgt aber Drift zu tatsächlich implementierten Operationen. Ohne verbindliche Versionen, Prüfergebnisse und Quellbezüge wird eine «verified»-Markierung zum unklaren Verwaltungsstatus. Die CDA-Architektur bevorzugt deshalb Ableitung aus vorhandenen Operationstabellen plus prüfbare, unabhängig begutachtete Evidence Records.

Automatische Laufzeitwahl unter allen passenden Workern. Sie kann theoretisch Flexibilität und Ausfallsicherheit erhöhen, erzeugt aber hohe Anforderungen an Policy, deterministische Wahl, Fehlersemantik, Zielidentität, Audit, Replay und Migration. Der aktuelle Entscheidungsweg wählt zuerst das enger definierte Deployment-Pinning und später allenfalls operatorgesteuerte Revisionen. Ein universeller dynamischer Scheduler ist nicht mit CDA0–CDA8 freigegeben.

Einschätzung: Die zunächst strengere Architektur ist nicht so bequem wie semantische Suche plus Sofort-Routing. Sie ist aber besser überprüfbar: Jeder Übergang von fachlichem Bedarf zu technischer Aktion hat eine benannte Quelle und einen expliziten Gate. Die Kosten liegen in Pflege von Contracts, Negativtests, Review und Governance. Dieser Aufwand ist nur dann wirtschaftlich, wenn mehrere Prozesse oder Zielsysteme tatsächlich von nachweisbaren Fähigkeiten und kontrolliertem Austausch profitieren.

17. Offene Entscheidungen und konkrete nächste Schritte

Der vorhandene Repository-Stand legt eine fachlich sinnvolle Reihenfolge nahe, ohne neue Architekturentscheidungen vorwegzunehmen:

1. **CDA2 fertigstellen.** Mindestens einen semantisch präzisen, geprüften Contract unter `workertype/contracts/` veröffentlichen; mit einer realen Worker-Type-Operation verknüpfen; positive und negative Fälle im Evidence Record mit Review sichern. Withdrawal und State-Projektion als getrennte Gate-Aufgaben entscheiden.
2. **Programm und Roadmap korrigieren.** PR #1401 ist gemerged, obwohl beide Planungsdokumente noch «in Review» nennen. Die Statuskorrektur darf ausdrücklich nicht als CDA2-Completion formuliert werden.
3. **CDA3 klein beginnen.** Erstes Account- oder Contact-Profil auswählen, semantische Grenzfälle beweisen und aus dem bestehenden Provider-Code ableiten; keine zweite Operationenquelle aufbauen.

4. **Graphdesign entscheiden.** ADR-0420 nur nach Provenienz-/Sichtbarkeitsprüfung akzeptieren; Graph-Abfragen standardmässig an eingeschränkten Knoten stoppen lassen.
5. **Binding und Runtime getrennt freigeben.** CDA6 beginnt mit reinem Deployment-Plan und Validierung; CDA7 erst nach einem tatsächlich unterstützten IAE-Enforcement-Pfad und Recovery-Nachweisen.
6. **CDA8 bis zum richtigen Zeitpunkt offenhalten.** Logische Slot-Identität in CDA6 berücksichtigen, aber Cutover-/Rollback-/Cross-Partition-Design erst nach einer angenommenen Entscheidung implementieren.

Für jede Runtime-Änderung gelten Regressionstests, Recovery-Vergleich von Live- und Replay-State, Fencing- und Idempotenztests, Prüfen der No-Allocation-Invariante und das repoübliche Build-/Test-/Race-/Vet-/Format-Gate. Die komplette Go-Suite wurde **für die Erstellung dieses dokumentarischen Whitepapers nicht ausgeführt**; es werden daher keine neuen Testergebnisse oder Produktionsfreigaben behauptet.

18. Fazit

CDA ist nicht bloss eine alternative Bezeichnung für bereits vorhandene Jobs oder eine schönere Darstellung von Integrationen. Die heute implementierten Bausteine liefern einen klaren Fortschritt: eine fachlich getrennte Business-Capability-Ebene, deklarative technische Claims, eine unabhängige Contract-Identität sowie überprüfbare Schema-, Digest- und Evidence-Infrastruktur. **Das eigentliche Capability-driven Routing ist noch Zielbild.** Gerade diese saubere Statusaussage ist der Ausgangspunkt für eine verlässliche Weiterentwicklung.

Eine belastbare nächste Version des Produkts wird nicht mit einem universellen «Kann alles»-Worker beginnen, sondern mit einer überprüften einzelnen Operation: exakte Semantik, unabhängige Evidenz, fehlerhafte Gegenbeispiele und danach ein autorisierter, replaybarer Ausführungspfad. Erst wenn dieser Nachweis erbracht ist, sollten die Graph- und Runtime-Projektionen ihn als verfügbar darstellen.

Quellen- und ADR-Verzeichnis

Primärquellen: `README.md`, `ROADMAP.md`, `AGENTS.md`, `docs/ARCHITECTURE.md`, `docs/architecture/invariants.md`, `CDA-Programm`, `IAE-Programm`, `Technischer CDA-Plan`, `ADR-Index`, `workertype`, `api/capability`, `engine` und PR #1401. Prüfzeitpunkt: 7. November 2026, Commit `8c6d819b2591af02a2b49b3d8ad362dc234df476`.

ADR	Gegenstand	Status am geprüften Stand
0305	Business Capabilities und Value Streams	Accepted / Partial
0308	Panorama-Bindung	Accepted / Landed
0203	Worker-Ausführungsmodell	Accepted / Landed
0208	Worker-Type-Pakete und Trust	Accepted / Partial

ADR	Gegenstand	Status am geprüften Stand
0480	Claims auf Operationen	Accepted / Partial
0485	Contract-Envelope	Accepted / Partial
0483	Evidence und Conformance Governance	Accepted / Partial
0420	Ein Graph, mehrere Projektionen	Proposed / Not started
0482	Capability-aware Binding und Routing	Proposed / Not started
0475	Identity-aware Execution	Proposed / Partial
0492	Controlled Worker-Rebinding	Proposed / Not started

Editionshinweis: Das bisherige `Atlas_Whitepaper_Capability_Driven_Automation.pdf` bleibt als historisches PDF bestehen. Diese Neuauflage wurde aus den aktuellen Repository-Quellen neu verfasst. Der binäre Inhalt der historischen PDF konnte über die verfügbare Repository-Schnittstelle nicht extrahiert werden; deshalb behauptet die Änderungsübersicht keinen zeilenweisen Vergleich mit der Altversion.